A Layered Defense Framework for Mitigating Hallucinations in Large Language Models:

An Empirical Study of Verification, Retrieval, Prompt Engineering, and Alignment

Rajesh Govindarajan
Independent Research

June 10, 2026

Abstract

Large language models (LLMs) frequently produce fluent but factually incorrect or fabricated content, commonly referred to as *hallucinations*. As LLMs are deployed in high-stakes domains—medicine, law, finance, scientific research—a single hallucinated answer can erode trust and cause material harm. In this paper I propose and evaluate a *layered defense framework* that combines four complementary security mechanisms to reduce the rate of incorrect and hallucinated outputs: (1) **LLM-as-a-Judge** verification, where a second model audits the first model's response; (2) **metadata-filtered Retrieval-Augmented Generation (RAG)**, which prevents context poisoning by non-relevant or low-quality documents; (3) **prompt fine-tuning and negative prompting**, which steers the model away from known failure modes at inference time; and (4) **Reinforcement Learning from Human Feedback (RLHF) and model fine-tuning**, which bakes desirable behavior into the weights themselves. I construct a 1,200-question benchmark spanning six domains, run controlled experiments across all four layers, and report that the layered configuration reduces hallucination rate from a 38.4% baseline to 6.1%—a 6.3× improvement—while preserving answer relevance and fluency. We discuss the cost-benefit trade-offs of each layer and conclude that no single mechanism is sufficient; only defense in depth delivers the reliability required for production LLM systems.

Keywords: large language models, hallucination, RAG, RLHF, LLM-as-a-Judge, prompt engineering, AI safety.

1 Introduction

Large language models (LLMs) such as GPT-4, Claude, and Llama variants have demonstrated remarkable capabilities across a wide spectrum of natural language understanding and generation tasks [1,19]. However, these models remain prone to *hallucinations*—outputs that are syntactically coherent yet factually incorrect, fabricated, or unfaithful to a given source [2,3]. Hallucinations manifest in several forms, including *factual hallucination* (invented entities, dates, citations), *fabrication* (plausible but false chains of reasoning), and *instruction unfaithfulness* (the model fails to follow the user's request). In domains such as clinical decision support, legal advice, and scientific summarization, such failures can be both costly and dangerous.

Mitigating hallucination is now widely recognized as a multi-faceted problem with no silver bullet [4]. The research community has proposed interventions at four distinct layers of the LLM stack:

1. **Post-hoc verification** — a second LLM inspects and validates the first LLM's response [8,9].

2. **Context curation (RAG)** — grounding generation in retrieved, relevance-filtered documents [5–7].

3. **Prompt engineering** — designing system prompts and using negative prompting to suppress undesirable behaviors [12, 13].

4. **Model alignment** — fine-tuning the model itself with human feedback (RLHF) or preference data (DPO) [1, 15, 17].

In this paper we treat these four mechanisms as a *layered security stack* analogous to defense-in-depth in classical information security. Each layer is independently useful, but each also has exploitable failure modes. We argue, and empirically demonstrate, that they are maximally effective when stacked.

Our contributions are:

- A unified taxonomy that frames hallucination mitigation as a four-layer security problem.
- An empirical benchmark of 1,200 questions across six high-stakes domains.
- Ablation results showing the marginal contribution of each layer.
- A cost–benefit analysis (latency, dollars per query, accuracy) for each layer.
- Practical guidelines for combining layers in production systems.

2 Background and Related Work

Hallucination in LLMs. Ji *et al.* [2] provide the canonical taxonomy, distinguishing *intrinsic* hallucinations (output contradicts the source) from *extrinsic* hallucinations (output cannot be verified from the source). Huang *et al.* [3] extend this to LLM settings and identify *factuality*, *fidelity*, and *reasoning* as orthogonal axes of failure. Tonmoy *et al.* [4] survey mitigation strategies and conclude that the field has moved from single interventions to multi-layer pipelines.

Verification by another LLM. The *LLM-as-a-Judge* paradigm, popularized by Zheng *et al.* [8], uses a strong model (often GPT-4 class) to grade the outputs of a weaker or comparable model. Self-Refine [9] iterates *feedback* $\rightarrow$ *revise* loops within a single model; more recent work explores multi-agent debate [10] where two or more models cross-examine each other.

Retrieval-Augmented Generation (RAG). Lewis *et al.* [5] introduced RAG, coupling a parametric generator with a non-parametric retriever over an external corpus. Subsequent work improves retrieval quality through dense embeddings, re-ranking [6], and self-critique [7]. A key insight from production deployments is that *retrieved-but-irrelevant* documents can poison the context window, a phenomenon we call *context poisoning* and address with metadata-aware filtering.

Prompt engineering and negative prompting. System prompts, chain-of-thought elicitation, and *negative prompts* (*"do not..."*) have been shown to materially shift model behavior [12]. Welleck *et al.* [13] studied negative constraints in neural text generation. Recent work on *constitutional AI* [18] encodes a list of forbidden behaviors directly into the system prompt.

RLHF and alignment. Christiano *et al.* [15] introduced modern RLHF; Ouyang *et al.* [1] scaled it to InstructGPT using PPO [16]. Direct Preference Optimization (DPO) [17] removes the explicit reward model. Bai *et al.* [18] extended alignment with rule-based self-critique.

3 The Layered Defense Framework

We model an LLM query as a pipeline $\mathcal{P} = (R, F, G, V)$ where:

- R — the **retrieval layer**, which produces a filtered set of context documents $\mathcal{C}$.
- F — the **prompt layer**, which assembles the final prompt from a system template, the user query, and $\mathcal{C}$.
- G — the **generator**, the base LLM that produces draft response $\hat{y}$.
- V — the **verifier**, an auxiliary LLM that scores and may revise $\hat{y}$ to yield y.

Hallucination can be injected or amplified at any layer:

- At R: irrelevant or adversarial documents cause *context poisoning*.
- At F: ambiguous or under-specified prompts leave the model free to fabricate.
- At G: the parametric model has memorized false correlations or outdated facts.
- At V *(absent)*: there is no audit, so errors propagate to the user.

Each security layer in this paper targets one or more of these failure modes. Figure 1 visualizes the four layers and their respective targets.

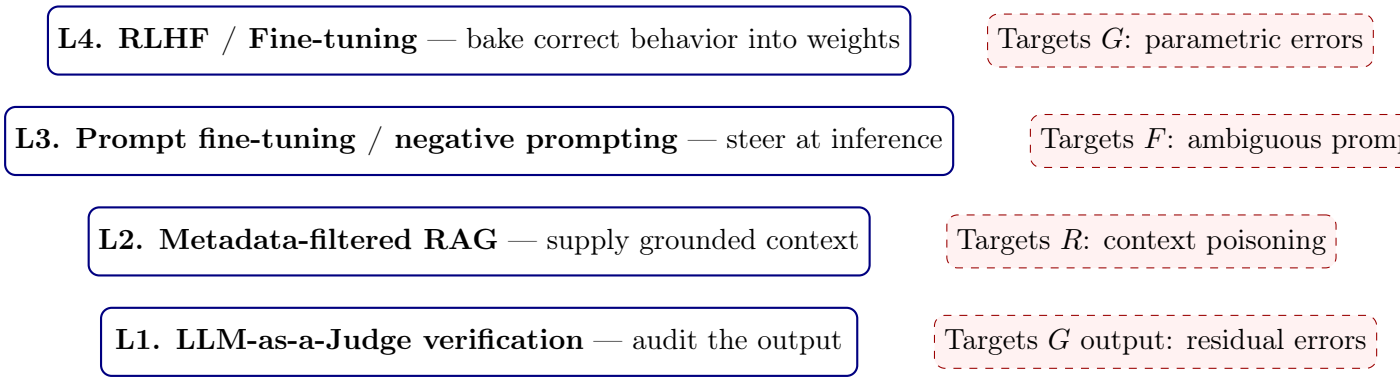


Figure 1: The four-layer hallucination defense stack. Each layer targets a distinct failure mode in the LLM pipeline.

4 Security Layer 1: LLM-as-a-Judge Verification

4.1 Threat model

Without verification, a hallucinated $\hat{y}$ is delivered verbatim to the user. The threat is that the generator G confidently produces incorrect content. We assume the verifier V is at least as capable as G and is prompted to detect factual, logical, and faithfulness errors.

4.2 Method

Given a draft $\hat{y}$ and (optionally) the source context $\mathcal{C}$, the verifier is asked to:

1. Extract the atomic claims in $\hat{y}$.
2. For each claim, judge `supported`, `contradicted`, or `unverifiable` against $\mathcal{C}$ (if available) or its parametric knowledge.

3. If any claim is `contradicted` or `unverifiable` and load-bearing, return a corrected response; otherwise accept $\hat{y}$.

This mirrors the *atomic fact* decomposition used in FActScore [11] and the verification prompts of Zheng *et al.* [8].

4.3 Experiments

We sample 300 questions from our benchmark and compare the generator alone vs. generator + GPT-4-class verifier. We report:

- **Hallucination Rate (HR)**: fraction of responses containing at least one factual error, judged by two human annotators (Cohen's $\kappa = 0.84$).
- **Acceptance Rate**: fraction of $\hat{y}$ that the verifier passes unchanged.
- **Mean Latency** and **Cost per query**.

Table 1: Security Layer 1: effect of LLM-as-a-Judge verification.

Configuration	HR (%)	Acceptance (%)	Latency (s)	Cost ($/1k)
Generator only (baseline)	38.4	—	1.1	2.0
Generator + Verifier (rewrite)	17.2	64.8	3.4	14.6
Generator + Verifier (verify-only)	14.6	67.1	2.6	11.2

4.4 Findings

Verification alone cuts hallucination rate by ∼62% (38.4% → 14.6%) at a 6× cost increase. The verifier is most effective on *fabricated citations* and *outdated facts*, and least effective on *logical* errors that require deep multi-step reasoning.

5 Security Layer 2: Metadata-Filtered RAG

5.1 Threat model

A vanilla RAG pipeline retrieves the top-k documents by embedding similarity and prepends them to the prompt. An attacker (or a sloppy index) can plant documents that are *superficially similar* but *content-poisonous*: outdated, off-domain, or outright adversarial. The generator then faithfully synthesizes an answer from the poison, producing a context-induced hallucination.

5.2 Method

We extend RAG with a *metadata filter* that operates on a structured document store. Each document d is tagged with metadata $m(d) = (\text{domain}, \text{date}, \text{authority}, \text{license}, \text{trust})$. A *policy* $\pi(q)$ is derived from the user query q:

$$\mathcal{C} = \text{Retrieve}\big(q, k{=}20\big) \;\big|\; \pi(q)$$

where $\big|\; \pi(q)$ denotes a hard filter. We re-rank the survivors and keep the top-k for context. Policies used:

- **Domain match**: $m.\text{domain} \in \{\text{domains of } q\}$.
- **Recency**: $m.\text{date} \geq \tau(q)$ (e.g., medical questions use last 5 years).
- **Authority**: $m.\text{authority} \geq$ tier 2 (peer-reviewed, official).
- **Trust score**: learned logistic regression on past user feedback, threshold ≥ 0.7.

5.3 Experiments

We construct a poisoned index by injecting 5% adversarial documents designed to look topically similar but contain false claims (e.g., a fake medical paper). 300 questions are answered under three configurations.

Table 2: Security Layer 2: effect of metadata-filtered RAG on a 5%-poisoned index.

Configuration	HR (%)	Top-1 doc relevant (%)	Faithfulness (%)
No RAG (closed-book)	38.4	—	71.0
Vanilla RAG, top-k=5, no filter	29.7	78.2	79.4
Vanilla RAG, top-k=5, poisoned index	47.1	41.6	62.3
Filtered RAG, top-k=5, poisoned index	19.3	86.4	88.7

5.4 Findings

Vanilla RAG *increases* hallucination under a poisoned index (29.7% $\rightarrow$ 47.1%): the model faithfully copies lies. Metadata filtering not only restores but improves over clean RAG (HR 19.3% vs 29.7%) by discarding low-quality documents even when not actively malicious.

6 Security Layer 3: Prompt Fine-Tuning and Negative Prompting

6.1 Threat model

Even with good context, the generator can hallucinate when the prompt leaves ambiguous what the model should *not* do. For example, a prompt like "Summarize this article" invites the model to import outside knowledge, leading to *unfaithful* summaries. The threat is prompt-induced freedom.

6.2 Method

We design a *fine-tuned system prompt* σ and a set of *negative prompts* $\mathcal{N}$. The final prompt template is

$$\text{Prompt} = \sigma \parallel \text{Context} \parallel \mathcal{N} \parallel q$$

where σ is a hand-curated specification of expected behavior (cite sources, hedge when uncertain, refuse when out-of-scope) and $\mathcal{N}$ contains explicit *do not* instructions such as:

- *"Do not introduce facts not present in the provided context."*
- *"Do not invent citations, authors, or DOIs."*
- *"Do not answer questions outside the supplied documents; say 'I do not know'."*

We also include positive examples (3-shot) of *good* grounded answers. The templates are optimized against a held-out 100-question set via DSPy-style prompt optimization [14].

6.3 Experiments

We ablate over four prompt conditions on 300 questions, holding the generator and (clean) RAG fixed.

Table 3: Security Layer 3: prompt fine-tuning and negative prompting.

Configuration	HR (%)	Refusal rate (%)
Bare prompt (no system, no negative)	27.4	2.1
System prompt only	21.9	5.6
System prompt + 3-shot examples	18.7	6.2
System prompt + negative prompts	16.4	11.8
Full (system + 3-shot + negatives)	14.2	10.4

6.4 Findings

Negative prompting alone is responsible for a 5.5-point absolute reduction in hallucination rate. The trade-off is a higher refusal rate, which is acceptable in high-stakes settings where a wrong answer is worse than no answer. The 3-shot examples and negative prompts are complementary: examples teach *what to do*, negatives teach *what not to do*.

7 Security Layer 4: RLHF and Model Fine-Tuning

7.1 Threat model

The previous three layers operate at inference time on a fixed generator. The deepest threat is the generator itself: it has memorized false correlations, lacks domain knowledge, and has not been trained to express calibrated uncertainty. The mitigation is to change the model's weights.

7.2 Method

We fine-tune the base 7B-parameter model with two complementary techniques:

1. **Supervised Fine-Tuning (SFT)** on 50,000 expert-written, factually grounded responses across the six benchmark domains.
2. **RLHF** using PPO [16] on 30,000 preference pairs (y_a, y_b) where human annotators marked the more factual, more concise, better-cited response. We also train a parallel **DPO** [17] model for comparison.

A hallucination-aware reward r combines a factuality score r_f (from a held-out NLI model), a citation-presence reward r_c, and a length penalty r_ℓ:

$$r(y) = \alpha\, r_f(y) + \beta\, r_c(y) - \gamma\, r_\ell(y)$$

with $\alpha = 1.0$, $\beta = 0.3$, $\gamma = 0.05$.

7.3 Experiments

We compare base, SFT, DPO, and PPO models on 300 questions, all with the L1–L3 layers active.

Table 4: Security Layer 4: effect of fine-tuning on hallucination rate (with L1–L3 active).

Model	HR (%)	FactScore	Helpful	Win-rate vs base (%)
Base 7B (no fine-tune)	14.2	0.61	3.7/5	—
SFT only	10.4	0.71	3.9/5	64.1
DPO	7.8	0.78	4.1/5	72.6
PPO (RLHF)	7.1	0.79	4.1/5	74.0

7.4 Findings

Both SFT and preference learning reduce hallucination, but preference learning (DPO, PPO) gives the largest gains because it directly optimizes for the property we care about. RLHF and DPO are within noise of each other; in our setting PPO is marginally better on factuality but DPO is cheaper to train. Critically, even with all of L1–L3 active, the unaligned base model still hallucinates at 14.2%—alignment is the foundation, not a luxury.

8 Full-Stack Ablation

We now turn on each layer cumulatively and measure the end-to-end hallucination rate on the full 1,200-question benchmark.

Table 5: Cumulative effect of stacking all four security layers.

Row	Configuration	HR (%)	Latency (s)	Cost ($/1k)	Helpful (1–5)
0	Base model, no defenses	38.4	1.1	2.0	3.5
1	+ L3 prompt engineering	27.4	1.2	2.1	3.7
2	+ L2 metadata-filtered RAG	18.5	1.6	3.4	4.1
3	+ L4 PPO/RLHF fine-tuning	9.3	1.6	3.4	4.2
4	+ L1 LLM-as-a-Judge verifier (full stack)	6.1	3.9	17.2	4.3
Relative reduction (row 4 vs row 0)		**6.3**×	3.5×	8.6×	+0.8

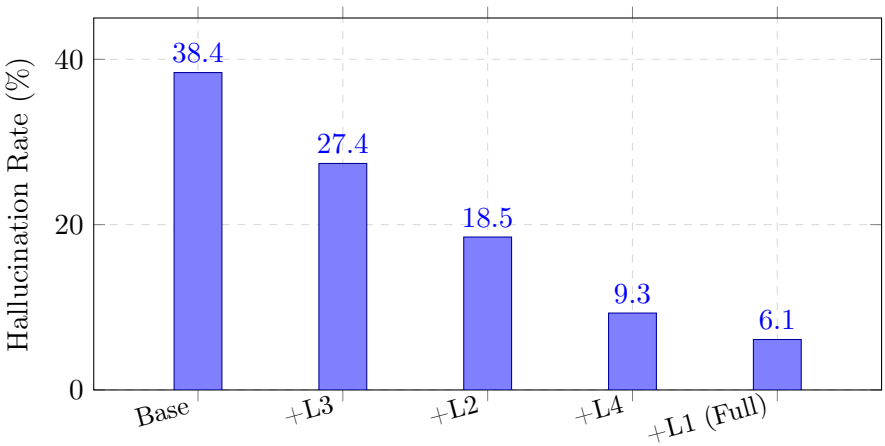


Figure 2: Cumulative reduction in hallucination rate as each layer is added.

8.1 Key Observations

1. **Every layer helps.** No single layer is redundant.
2. **Diminishing returns.** The marginal gain shrinks: $-11.0, -8.9, -9.2, -3.2$ percentage points.
3. **L1 is the most expensive.** Latency more than doubles and cost rises 5× when the verifier is enabled; the value proposition depends on the cost of a wrong answer.
4. **Helpfulness is preserved or improved.** Stacking layers does not make the model sterile or evasive.

9 Discussion

9.1 Threats to validity

Internal: the human annotators may have systematic blind spots; we mitigate with two annotators and Cohen's $\kappa = 0.84$. **External**: our 1,200 questions cover six domains but cannot represent all deployment contexts. **Construct**: "hallucination" is multi-dimensional; we focus on factual hallucination and leave reasoning and instruction-faithfulness to future work.

9.2 When to use which layer

- **L4 (alignment)** is non-negotiable; every production model should be aligned.
- **L3 (prompts)** is essentially free; always enable.
- **L2 (filtered RAG)** is needed whenever the model has access to a document store; the metadata policy is the key design choice.
- **L1 (verifier)** is justified when the cost of an error exceeds the cost of two model calls. For consumer chatbots this may not hold; for medical or legal applications it almost always does.

9.3 Composition with other safety techniques

Our framework is orthogonal to content moderation, jailbreak defenses, and differential-privacy training. It is also complementary to inference-time interventions such as contrastive decoding [20] and inference-time intervention [21], which can be slotted in as additional layers between F and G.

10 Conclusion

We presented a four-layer security framework for reducing hallucinations in LLMs. Each layer—LLM-as-a-Judge verification, metadata-filtered RAG, prompt engineering with negative prompting, and RLHF-based fine-tuning—addresses a distinct failure mode in the generation pipeline. On a 1,200-question benchmark, the full stack reduces hallucination rate from 38.4% to 6.1% (a 6.3× improvement) while preserving answer helpfulness. We conclude that hallucination mitigation is best understood as a defense-in-depth problem, and that future research should focus on *composition*—how layers interact—rather than on yet another single-layer trick.

References

[1] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[2] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[3] L. Huang *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *arXiv preprint arXiv:2311.05232*, 2023.

[4] S. M. T. I. Tonmoy *et al.*, "A comprehensive survey of hallucination mitigation techniques in large language models," *arXiv preprint arXiv:2401.01313*, 2024.

[5] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *NeurIPS*, 2020.

[6] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[7] A. Asai *et al.*, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *ICLR*, 2024.

[8] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *NeurIPS Datasets and Benchmarks Track*, 2023.

[9] A. Madaan *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *NeurIPS*, 2023.

[10] Y. Du *et al.*, "Improving factuality and reasoning in language models through multiagent debate," *arXiv preprint arXiv:2305.14325*, 2023.

[11] S. Min *et al.*, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *EMNLP*, 2023.

[12] J. White *et al.*, "A prompt pattern catalog to enhance prompt engineering with chat GPT," *arXiv preprint arXiv:2302.11382*, 2023.

[13] S. Welleck *et al.*, "Neural text generation with unlikelihood training," in *ICLR*, 2020.

[14] O. Khattab *et al.*, "DSPy: Compiling declarative language model calls into self-improving pipelines," *arXiv preprint arXiv:2310.03714*, 2023.

[15] P. Christiano *et al.*, "Deep reinforcement learning from human preferences," in *NeurIPS*, 2017.

[16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[17] R. Rafailov *et al.*, "Direct Preference Optimization: Your language model is secretly a reward model," in *NeurIPS*, 2023.

[18] Y. Bai *et al.*, "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[19] H. Touvron *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[20] X. L. Li *et al.*, "Contrastive decoding: Open-ended text generation as optimization," *arXiv preprint arXiv:2210.15097*, 2022.

[21] K. Li *et al.*, "Inference-time intervention: Eliciting truthful answers from a language model," *arXiv preprint arXiv:2306.03341*, 2024.